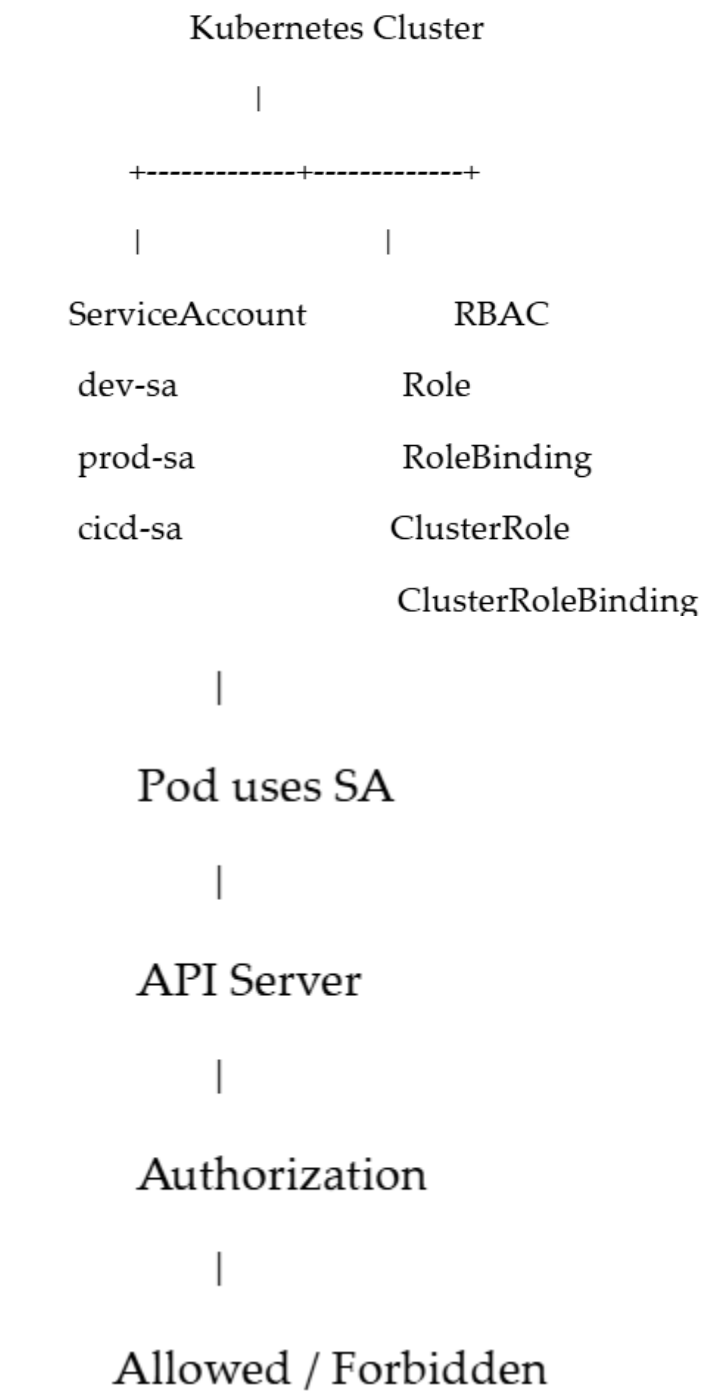
Kubernetes ServiceAccount & RBAC — Hands-on Practicals

We will build this lab:



Lab 1 — Create a ServiceAccount

Create namespace:

kubectl create namespace rbac-lab

Create ServiceAccount:

kubectl create serviceaccount dev-sa -n rbac-lab

Verify:

kubectl get serviceaccount -n rbac-lab

Expected:

NAME SECRETS AGE

default 0 ...

dev-sa 0 ...

Describe:

kubectl describe serviceaccount dev-sa -n rbac-lab

Important

Modern Kubernetes does **not automatically create a long-lived Secret token** for every ServiceAccount.

Instead, Pods normally receive a projected service-account token.

Lab 2 — Create a Pod using ServiceAccount

Create: sa-test.yaml

apiVersion: v1

```
kind: Pod
metadata:
  name: sa-test
  namespace: rbac-lab
spec:
  serviceAccountName: dev-sa
  containers:
  - name: nginx
    image: nginx
```

Apply:

```
kubectl apply -f sa-test.yaml
```

Check:

```
kubectl get pod -n rbac-lab
```

Check which SA the Pod uses:

```
kubectl get pod sa-test -n rbac-lab \
  -o jsonpath='{.spec.serviceAccountName}'
```

Output:

```
dev-sa
```

Lab 3 — Understand the ServiceAccount Token

Enter the Pod:

kubectl exec -it sa-test -n rbac-lab -- bash

Check:

ls /var/run/secrets/kubernetes.io/serviceaccount/

You should see files such as:

ca.crt

namespace

token

Check namespace:

cat /var/run/secrets/kubernetes.io/serviceaccount/namespace

Check token existence:

ls -l /var/run/secrets/kubernetes.io/serviceaccount/token

The token allows the application to authenticate to the Kubernetes API server.

Lab 4 — ServiceAccount without RBAC

At this point, dev-sa has **no special permissions**.

Test:

```
kubectl auth can-i get pods \
  --as=system:serviceaccount:rbac-lab:dev-sa \
  -n rbac-lab
```

Expected:

no

Test:

```
kubectl auth can-i create pods \
  --as=system:serviceaccount:rbac-lab:dev-sa \
  -n rbac-lab
```

Expected:

no

This demonstrates:

ServiceAccount ≠ Permission

The ServiceAccount provides an **identity**.

RBAC provides the **permissions**.

Lab 5 — Create a Role

Create: pod-reader-role.yaml

```
apiVersion: rbac.authorization.k8s.io/v1
kind: Role
metadata:
  name: pod-reader
  namespace: rbac-lab
rules:
- apiGroups: [""]
```

resources:

- pods

verbs:

- get

- list

- watch

Apply:

kubectl apply -f pod-reader-role.yaml

Check:

kubectl get role -n rbac-lab

Describe:

kubectl describe role pod-reader -n rbac-lab

Lab 6 — RoleBinding

Now connect:

ServiceAccount

|

| RoleBinding

↓

Role

Create:

```
apiVersion: rbac.authorization.k8s.io/v1
kind: RoleBinding
metadata:
  name: dev-sa-pod-reader
  namespace: rbac-lab
subjects:
- kind: ServiceAccount
  name: dev-sa
  namespace: rbac-lab
roleRef:
  kind: Role
  name: pod-reader
  apiGroup: rbac.authorization.k8s.io
```

Apply:

```
kubectl apply -f rolebinding.yaml
```

Lab 7 — Test RBAC

Now:

```
kubectl auth can-i get pods \
  --as=system:serviceaccount:rbac-lab:dev-sa \
  -n rbac-lab
```

Expected:

yes

Test:

```
kubectl auth can-i list pods \
  --as=system:serviceaccount:rbac-lab:dev-sa \
  -n rbac-lab
```

Expected:

yes

But:

```
kubectl auth can-i delete pods \
  --as=system:serviceaccount:rbac-lab:dev-sa \
  -n rbac-lab
```

Expected:

no

And:

```
kubectl auth can-i get deployments \
  --as=system:serviceaccount:rbac-lab:dev-sa \
  -n rbac-lab
```

no

Why?

Because our Role only says:

resources:

- pods

verbs:

- get

- list

- watch

RBAC follows the principle:

Only explicitly granted permissions are allowed.

Lab 8 — Give ServiceAccount Deployment Permissions

Modify Role:

```
apiVersion: rbac.authorization.k8s.io/v1
kind: Role
metadata:
  name: developer-role
  namespace: rbac-lab
rules:
```

```
 # Pods

  - apiGroups: [""]
    resources:
    - pods
    verbs:
    - get
    - list
    - watch
# Deployments
- apiGroups: ["apps"]
  resources:
  - deployments
  verbs:
  - get
  - list
  - watch
  - create
  - update
```

Apply:

kubectl apply -f developer-role.yaml

Test:

```
kubectl auth can-i create deployments \
  --as=system:serviceaccount:rbac-lab:dev-sa \
  -n rbac-lab
```

Expected:

yes

But:

```
kubectl auth can-i delete deployments \
  --as=system:serviceaccount:rbac-lab:dev-sa \
 -n rbac-lab
```

Expected:

no

Lab 9 — Secret Access

This is an **important real-world practical**.

Add:

```
- apiGroups: [""]
  resources:
  - secrets
  verbs:
   - get
```

```
  - list
```

Now:

```
kubectl auth can-i get secrets \
  --as=system:serviceaccount:rbac-lab:dev-sa \
  -n rbac-lab
```

Expected:

```
yes
```

This demonstrates why you should **not unnecessarily grant Secret permissions**.

For example, avoid:

```
resources:
- "*"


verbs:
- "*"
```

That's essentially an administrative-level permission set and violates least privilege.

Lab 10 — ClusterRole vs Role

This is one of the most important interview concepts.

Role

Namespace-scoped:

rbac-lab

|

+-- Role

|

+-- pods

+-- deployments

ClusterRole

Cluster-wide RBAC object.

Example:

```
apiVersion: rbac.authorization.k8s.io/v1
kind: ClusterRole
metadata:
  name: pod-reader-cluster
rules:
- apiGroups: [""]
  resources:
  - pods
  verbs:
  - get
  - list
```

```
  - watch
```

Create:

```
kubectl apply -f clusterrole.yaml
```

Lab 11 — ClusterRoleBinding

Bind it to the ServiceAccount:

```
apiVersion: rbac.authorization.k8s.io/v1
kind: ClusterRoleBinding
metadata:
  name: dev-sa-cluster-reader
subjects:
- kind: ServiceAccount
  name: dev-sa
  namespace: rbac-lab
roleRef:
  kind: ClusterRole
  name: pod-reader-cluster
  apiGroup: rbac.authorization.k8s.io
```

Apply:

```
kubectl apply -f clusterrolebinding.yaml
```

Now the ServiceAccount can read Pods across namespaces.

Test:

```
kubectl auth can-i get pods \
  --as=system:serviceaccount:rbac-lab:dev-sa \
  --all-namespaces
```

Expected:

```
yes
```

This illustrates:

ClusterRole

```
ClusterRole
    |
    ↓
ClusterRoleBinding
    |
    ↓
ServiceAccount
    |
    ↓
Multiple namespaces
```

Lab 12 — Role + ClusterRole Comparison

Feature	Role	ClusterRole
Scope	Namespace	Cluster

Feature	Role	ClusterRole
Pods	Yes	Yes
Deployment	Yes	Yes
Nodes	No namespace	Yes
Namespace	No	Yes
Cluster-wide	No	Yes
Binding	RoleBinding	ClusterRoleBinding
Least privilege	Excellent	Use carefully

Lab 13 — Developer vs Admin

Create two ServiceAccounts:

kubectl create sa developer -n rbac-lab

kubectl create sa admin-user -n rbac-lab

Developer:

developer

|

↓

Role

```
  |
  +-- get pods
  +-- list pods
```

```
  +-- get deployments
  +-- update deployments
```

```
Admin:
admin-user
    |
    ↓
ClusterRole
    |
    +-- broad permissions
```

```
Test:
kubectl auth can-i delete pods \
 --as=system:serviceaccount:rbac-lab:developer \
 -n rbac-lab
```

versus:

```
kubectl auth can-i delete pods \
  --as=system:serviceaccount:rbac-lab:admin-user \
  -n rbac-lab
```

This is a good demonstration of **least privilege**.

Lab 14 — CI/CD ServiceAccount

This is a very practical DevOps scenario.

Imagine:

```
GitHub Actions/Jenkins
        |
        ↓
Kubernetes
        |
        ↓
Deploy application
```

Create:

```
kubectl create sa cicd-sa -n rbac-lab
```

Create a Role that allows:

get deployments

create deployments

update deployments

patch deployments

get services

create services

update services

patch services

get pods

list pods

Then:

CI/CD

|

| authentication

↓

cicd-sa

|

| authorization

↓

↓

RoleBinding

|

↓

deployment permissions

This is much safer than giving CI/CD:

cluster-admin

Lab 15 — Debug an RBAC "Forbidden" Error

This is an excellent interview practical.

Suppose:

```
kubectl get pods
```

returns:

```
Error from server (Forbidden):
pods is forbidden
```

Check identity:

```
kubectl auth whoami
```

Check permission:

```
kubectl auth can-i get pods
```

For a ServiceAccount:

```
kubectl auth can-i get pods \
  --as=system:serviceaccount:rbac-lab:dev-sa \
```

-n rbac-lab

Check Role:

kubectl get role -n rbac-lab

Check RoleBinding:

kubectl get rolebinding -n rbac-lab

Describe:

kubectl describe rolebinding dev-sa-pod-reader \

-n rbac-lab

Then verify:

ServiceAccount name

↓

RoleBinding subject

↓

RoleRef

↓

Role

↓

resources

↓

verbs

Lab 16 — The Most Important RBAC Matrix

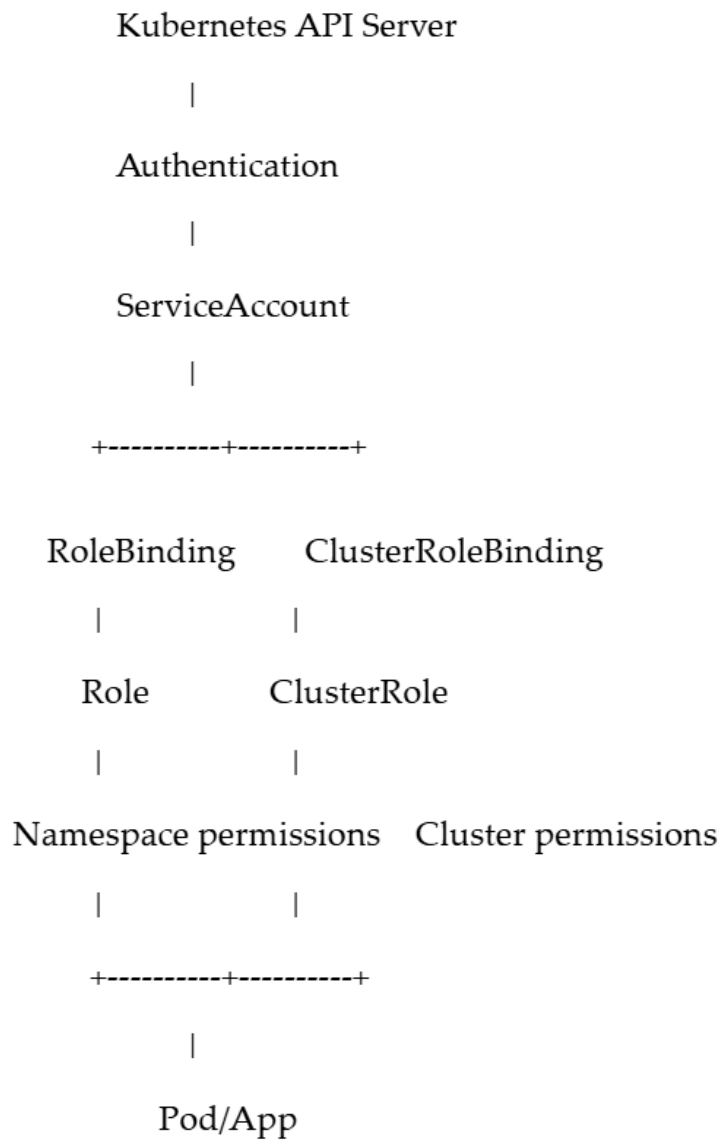
Practice these commands:

```
kubectl auth can-i get pods \
  --as=system:serviceaccount:rbac-lab:dev-sa \
  -n rbac-lab
kubectl auth can-i create pods \
  --as=system:serviceaccount:rbac-lab:dev-sa \
  -n rbac-lab
kubectl auth can-i delete pods \
  --as=system:serviceaccount:rbac-lab:dev-sa \
  -n rbac-lab
kubectl auth can-i get secrets \
  --as=system:serviceaccount:rbac-lab:dev-sa \
  -n rbac-lab
kubectl auth can-i get nodes \
  --as=system:serviceaccount:rbac-lab:dev-sa
kubectl auth can-i '*' '*' \
  --as=system:serviceaccount:rbac-lab:dev-sa
```

The last one is particularly useful for determining whether an identity effectively has broad permissions.

Lab 17 — Final Real-World Architecture

A good production design looks like:



Remember this formula

ServiceAccount = WHO

Role = WHAT

RoleBinding = CONNECT WHO + WHAT

For example:

dev-sa

+

pod-reader

+

RoleBinding

dev-sa can read Pods

And:

cicd-sa

+

deployment-role

+

RoleBinding

=

CI/CD can deploy applications